

Validação cruzada: do rodízio ao esquema certo

k-fold, estratificação, grupos e tempo — sem vazamento (Aulas 06 e 07 juntas)

De onde viemos: um corte só é jogar a moeda

Na Aula 05 vimos: avaliar com **uma única divisão treino/teste** depende da sorte do corte — a nota balança. Hoje resolvemos isso.

- **O problema (recap)** — mesmo modelo, mesma base: só mudando a semente do corte, a acurácia vai de 0,82 a 0,76.
- **A ideia de hoje** — em vez de UM corte, fazer VÁRIOS, testar em cada um e tirar a média — estimativa estável.
- **E mais** — escolher o esquema certo (estratificado, por grupo, temporal) e usá-lo sem vazamento (Pipeline).

Objetivo da aula

Estimar o desempenho de um modelo de forma **estável e honesta**, com a **validação cruzada**, e escolher o esquema adequado a cada tipo de dado.

O CAMINHO

- **A ideia** — k-fold: o rodízio que aproveita todo o dado.
- **O esquema certo** — estratificado, por grupo e temporal.
- **Sem vaziar** — CV com Pipeline e Nested CV.

A ANALOGIA DE HOJE

- **Prova em vários dias** — julgar um aluno por uma prova só é injusto (dia bom/ruim). Média de várias provas é mais justa.
- A validação cruzada é isso com os dados.

Da Aula 05 (hold-out) para a estimativa estável: trocar a sorte de um corte pela média de vários.

PARTE 1

A ideia: k-fold

O rodízio que aproveita todo o dado.

1

Uma divisão só engana

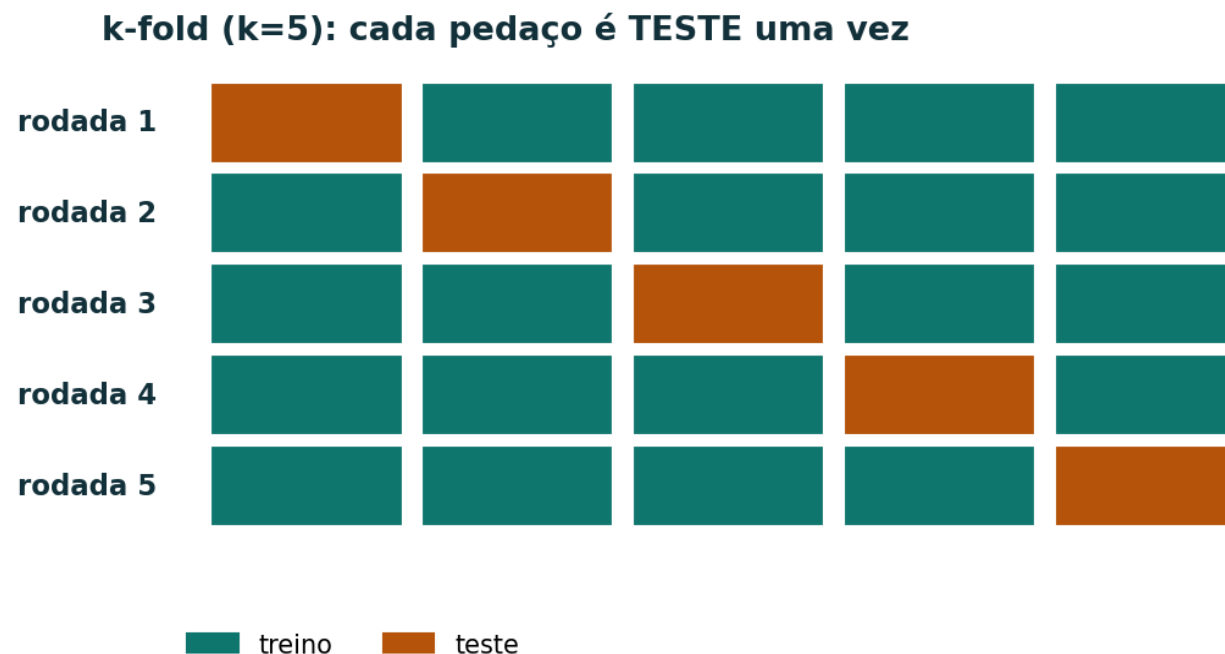
Com um corte, você tem **um número**, e ele tem **variância**: um teste 'fácil' infla, um 'difícil' derruba.

semente do corte	acurácia no teste
0	0,82
1	0,76
2	0,85
3	0,79

- O PONTO**
- Mesmo modelo, mesmo dado.
 - Só mudou o sorteio.
 - Qual desses é o desempenho 'de verdade'? Nenhum — é preciso a MÉDIA.

A validação cruzada troca esse número único pela média de vários cortes.

k-fold: o rodízio



Divide o dado em k pedaços iguais. Em cada rodada, um pedaço é TESTE e os outros $k-1$ são TREINO. Todo dado é testado uma vez — e treinado $k-1$ vezes.

k-fold na mão (k = 5)

Rodamos 5 vezes; cada rodada dá uma acurácia. O resultado é a **média** e o **desvio-padrão** delas.

rodada	1	2	3	4	5
acurácia	0,80	0,90	0,70	0,85	0,75

RESULTADO

$$\text{média} = (0,80+0,90+0,70+0,85+0,75)/5 = 0,80 \cdot \text{desvio} \approx 0,07 \rightarrow 0,80 \pm 0,07$$

Reporta-se SEMPRE os dois: a média (o desempenho esperado) e o desvio (o quanto ele oscila entre os cortes).

O que a validação cruzada entrega

- **Estimativa estável** — a média de k cortes varia muito menos que um corte só.
- **A incerteza, de brinde** — o desvio-padrão diz se o modelo é consistente ou instável entre os cortes.
- **Todo dado é aproveitado** — cada exemplo serve de teste uma vez e de treino $k-1$ vezes — importante com pouco dado.
- **Serve para comparar** — dois modelos com a mesma CV: quem tem média melhor (e desvio parecido) vence.

Escolhendo o k

k pequeno (ex.: 5)

- **Rápido** — só 5 treinos; teste de 20% cada.
- **Treino menor** — cada modelo vê 80% dos dados.
- **Padrão** — bom equilíbrio para a maioria dos casos.

k grande (ex.: 10)

- **Treino maior** — cada modelo vê 90% — estimativa menos enviesada.
- **Mais caro** — 10 treinos; e o desvio entre dobras pode subir.
- **Uso** — quando há dado suficiente e você quer capricho.

Regra prática: $k = 5$ ou 10 . Muito dado $\rightarrow 5$ já basta; pouco dado $\rightarrow 10$ (ou LOO) aproveita mais.

k-fold no scikit-learn

```
from sklearn.model_selection import cross_val_score, KFold
scores = cross_val_score(modelo, X, y, cv=5, scoring="accuracy")
print(scores)                # [0.80 0.90 0.70 0.85 0.75]
print(scores.mean(), scores.std()) # 0.80 0.07
```

cv=5 faz o rodízio sozinho. O resultado honesto é média $\pm$ desvio — nunca um número solto.

Quem define treino e teste? O cv, sozinho

Você não separa nada à mão: o **cv=5** corta o X, y em 5 pedaços e faz o rodízio automaticamente.

rodada	teste	treino
1	pedaço 1	pedaços 2, 3, 4, 5
2	pedaço 2	pedaços 1, 3, 4, 5
3	pedaço 3	pedaços 1, 2, 4, 5
4	pedaço 4	pedaços 1, 2, 3, 5
5	pedaço 5	pedaços 1, 2, 3, 4

Esse X, y é só o bloco de treino/validação — o teste final fica SEPARADO e é tocado uma única vez no fim.

PARTE 2

O esquema certo

Nem todo dado pode ser sorteado.

2

StratifiedKFold: preserva as classes

Se a classe é **rara**, um sorteio pode deixar uma dobra **sem nenhum positivo** — a métrica quebra.

100 exemplos, 10 positivos (10%)	dobra 1	dobra 2	dobra 3	dobra 4	dobra 5
k-fold aleatório: positivos na dobra	4	0	3	1	2
StratifiedKFold: positivos na dobra	2	2	2	2	2

O estratificado mantém a proporção (10%) em CADA dobra → recall/precisão fazem sentido em todas. Em classificação, é o padrão.

E quando a classe rara é rara demais?

Estratificar **não cria positivos**: só distribui os que existem. Logo, **k não pode passar do nº de positivos**.

Funciona: 8 positivos, k=5

- $8 \div 5 \approx 1,6$ por dobra
- Distribui 2, 2, 2, 1, 1
- **Toda dobra tem positivo** — recall/precisão fazem sentido em todas.

Quebra: 3 positivos, k=5

- **3 positivos para 5 dobras**
- **2 dobras ficam com ZERO** — métrica indefinida (0/0).
- **Saída** — baixe o k ($k=3 \rightarrow 1$ por dobra) ou use RepeatedStratifiedKFold.

Regra de bolso: $k \leq \text{nº de exemplos da classe mais rara}$. Com 1 só positivo, nem $k=2$ dá — precisa de mais dado.

Leave-one-out

Leave-One-Out (LOO)

- **Quando** — pouquíssimos dados — aproveita ao máximo.
- **Custo** — n treinos; caro e com estimativa 'nervosa'.

LOO $\times$ k-fold, na mão ($n = 10$)

LOO é **k-fold com $k = n$** : cada exemplo, sozinho, vira teste uma vez.

k-fold ($k = 5$)

- **5 rodadas**
- **teste = 2 exemplos** — treino = 8.
- **5 modelos, 5 notas** — cada nota já mede em 2 pontos $\rightarrow$ mais estável.

Leave-One-Out ($k = 10$)

- **10 rodadas**
- **teste = 1 exemplo** — treino = 9.
- **10 modelos, 10 notas** — cada nota é 0 ou 1 — só a média faz sentido.

CV repetida

RepeatedKFold

- **O que é** — repete o k-fold várias vezes, com sorteios diferentes.
- **Por quê** — a média de repetições reduz o efeito da sorte de UM sorteio.
- **Uso** — quando você quer uma estimativa ainda mais estável.

RepeatedKFold, na mão

O **RepeatedKFold** roda o k-fold várias vezes, cada vez com um **sorteio diferente**, e tira a média de TODAS as notas.

repetição (sorteio diferente)	as 5 notas do k-fold	média
repetição 1	$0,80 \cdot 0,90 \cdot 0,70 \cdot 0,85 \cdot 0,75$	0,80
repetição 2	$0,85 \cdot 0,80 \cdot 0,75 \cdot 0,90 \cdot 0,85$	0,83
repetição 3	$0,75 \cdot 0,80 \cdot 0,80 \cdot 0,70 \cdot 0,85$	0,78

RESULTADO

média das 15 notas = 0,80 · sem repetir, você poderia ter ficado com 0,78 ou 0,83 — só pela sorte do sorteio

5-fold repetido 3x = 15 notas. Uma repetição só balança (0,78 a 0,83); repetir e mediar espreme a sorte e firma a estimativa.

GroupKFold: o grupo inteiro de um lado só

A REGRA

todas as linhas de um mesmo grupo ficam JUNTAS — ou todas no treino, ou todas no teste, nunca dos dois lados

```
from sklearn.model_selection import GroupKFold, cross_val_score
gkf = GroupKFold(n_splits=5)
cross_val_score(modelo, X, y, cv=gkf, groups=paciente) # 'groups' = o id do grupo
```

Use sempre que houver repetição por pessoa/loja/sensor. Sem isso, a acurácia parece ótima e desaba na vida real.

O problema do GRUPO: o modelo decora a pessoa

Você tem **6 exames de 3 pacientes** (2 de cada). O sorteio aleatório manda exames do **mesmo paciente** para os dois lados.

paciente	exame	sorteio
P1	exame 1	treino
P1	exame 2	TESTE ← P1 já visto
P2	exame 3	treino
P2	exame 4	TESTE ← P2 já visto
P3	exame 5	treino
P3	exame 6	TESTE ← P3 já visto

A CONTA QUE DENUNCIA

- **Validação: 0,95** — parece ótimo.
- **Paciente NOVO (P9): 0,68** — desaba na vida real.
- **Por quê?** — decorou QUEM, não a doença.

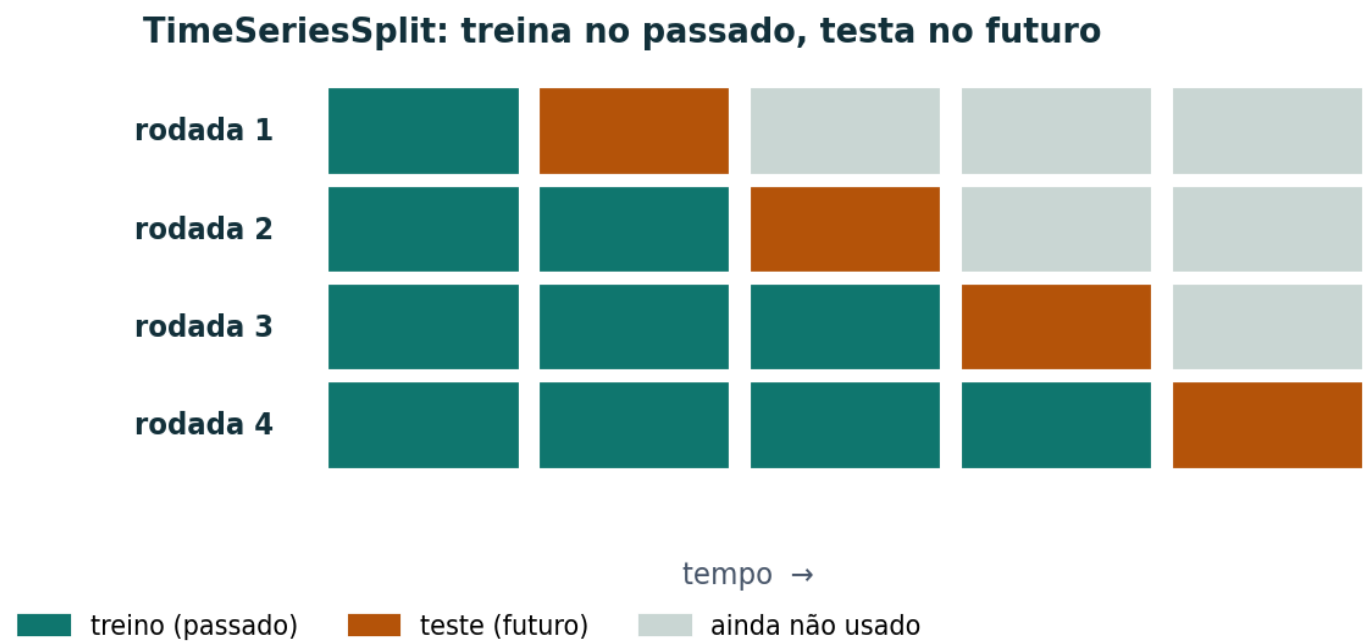
Em produção chegam SEMPRE pacientes novos. O teste tem de imitar isso — nenhum paciente dos dois lados. É o vazamento da Aula 05, pela identidade do grupo.

O tempo: por que o aleatório vaza o futuro

Com data, sortear as linhas coloca o **futuro no treino** — o modelo 'vê' o que ainda não aconteceu (vazamento temporal, da Aula 05).

- **O erro** — k-fold aleatório mistura datas: treina com dezembro para prever novembro.
- **A correção** — cortar por tempo: treinar sempre no passado e testar no futuro.
- **O esquema** — TimeSeriesSplit — janelas que crescem no tempo.

TimeSeriesSplit: treina no passado, testa no futuro



A cada rodada o treino cresce (passado) e o teste é o próximo trecho (futuro). Nunca se testa em algo anterior ao treino.

Um corte temporal ou vários?

Treinar 2000–2015 e segurar 2016–2021 é um **hold-out temporal**: o TimeSeriesSplit com um corte só.

Hold-out temporal (1 corte)

- **treino 2000–2015 · teste 2016–2021**
- **uma estimativa** — barata e realista.
- **o risco** — 2016–2021 pode ser um período atípico → a nota depende da sorte daquele pedaço.

TimeSeriesSplit (vários)

- **treina passado, testa o próximo trecho — repetido**
- **várias estimativas** — média mais estável; testa épocas diferentes.
- **a 'sobra' de dado** — os mais antigos nunca são teste — é inerente ao tempo, não desperdício.

Os dois só ESTIMAM. Aprovado o modelo, re-treine com TODOS os dados (2000–2021) para colocar em produção e prever 2022.

PARTE 3

Usar sem vazar

Pipeline dentro da CV e a maldição do vencedor.

3

CV com Pipeline (o certo)

O pré-processamento (padronizar, imputar) tem de ser feito **dentro de cada dobra**, só com o treino daquela dobra.

```
from sklearn.pipeline import Pipeline
pipe = Pipeline([("prep", StandardScaler()), ("clf", modelo)])
cross_val_score(pipe, X, y, cv=5)  # o fit do scaler acontece SÓ no treino de cada dobra
```

Sem o Pipeline, padronizar antes da CV usa as médias do teste → vazamento e nota otimista (recap da Aula 05).

Qual esquema usar? Tabela de decisão

Situação do dado	Esquema	Por quê
Classificação (padrão)	StratifiedKFold	mantém a proporção das classes em cada dobra
Regressão / dado comum	KFold (k=5 ou 10)	o rodízio simples já basta
Repetição por pessoa/loja/sensor	GroupKFold	o grupo inteiro de um lado só (sem vaziar)
Dados com data (série temporal)	TimeSeriesSplit	treina no passado, testa no futuro
Pouquíssimos dados	LOO / RepeatedKFold	aproveita ao máximo cada exemplo

Erros comuns e boas práticas

Erros comuns

- Reportar um corte só (sem CV).
- Padronizar/imputar antes da CV (vaza).
- k-fold aleatório com grupos ou com tempo.
- Escolher e medir na mesma CV.
- Esquecer o desvio-padrão.

Boas práticas

- CV com $k=5/10$; estratificado em classificação.
- Sempre dentro de um Pipeline.
- GroupKFold com repetição; TimeSeriesSplit com data.
- Nested CV ao ajustar hiperparâmetros.
- Reportar média $\pm$ desvio.

Na próxima: busca de hiperparâmetros (grid/random search) — que vive DENTRO da validação cruzada.